LOT Pager — LOT API Connector

Document: LOT-HW-01 / API · **Rev:** A (2026-11-13) · **Status:** design, not yet implemented

How the device talks to lot-systems.com. Written against the current server code so it can be built in the next session.

1. What exists today (read from this repo)

Fact	Where
Fastify server; routes registered with prefixes `/auth`, `/api`, `/admin-api`, `/api/public`	`src/server/routes/index.ts`
User API auth = session cookie → `req.user`; un-authed requests get 401	`src/server/index.ts:~280–320`
Log entries: model Log (`userId`, `text`, `event`, `metadata` JSONB, `context` JSONB)	`src/server/models/log.ts`
`POST /api/logs` creates a log (`text`, `event`, `metadata`)	`src/server/routes/api.ts:~1563`
Log tab only shows events in an allow-list `displayableEvents`	`src/server/routes/api.ts:~1082`
AI/pattern notices already generated: `generateContextualPrompts()` → `{type,title,message,priority}`	`src/server/utils/contextual-prompts.ts`, `GET /api/contextual-prompts`
Weather cache model `WeatherResponse`	`src/server/models/weather-response.ts`
Background jobs	`src/server/scheduled-jobs.ts`

Cookies don't suit a device, so the device gets its **own token** and its own route group.

2. New data model

```
devices
  id            uuid pk
  userId        uuid → users.id (cascade)
  serial        text unique         -- laser-etched serial, e.g. LOT-P1-0042
  publicKey     text                -- ATECC608B P-256 public key (factory)
  tokenHash     text                -- sha256 of device token
  name          text default 'LOT Pager'
  firmware      text
  lastSeenAt    timestamptz
  battery       int
  createdAt / updatedAt

device_notices
  id            uuid pk
  userId        uuid
  text          varchar(84)         -- fits 6 × 14 chars on screen
  source        text                -- 'contextual' | 'ai' | 'manual'
  priority      int
  deliveredAt   timestamptz null
  copiedAt      timestamptz null
  createdAt
```

3. Routes — `/api/device/*`

Registered as a separate Fastify plugin **outside** the cookie-guarded user API, with its own `onRequest` hook: `Authorization: Bearer <device token>` → look up `tokenHash` → set `req.device` and `req.user`. Rate limit: 60 req / 10 min per device.

Method & path	Auth	Purpose
`POST /api/device/pair/start`	user cookie	User clicks "Add LOT Pager" in Settings → returns 8-char pair code (10 min TTL)
`POST /api/device/pair/finish`	none (code + serial + ATECC signature)	Device exchanges code → long-lived device token (shown once)
`GET /api/device/inbox?since=<cursor>`	device	Returns undelivered notices (max 3), marks delivered. ETag/304 when empty
`POST /api/device/copy`	device	Copy button → creates **Log** row `event: 'device_copy'`
`POST /api/device/session`	device	Compressed session batch (CBOR + heatshrink): sensor series, battery, errors
`POST /api/device/photo`	device	JPEG (≤ 64 KB) attached to a copy log
`GET /api/device/ota`	device	Latest signed firmware manifest
`DELETE /api/device/:id`	user cookie	Unpair from Settings

3.1 Copy → Log tab

```
POST /api/device/copy
Authorization: Bearer lotd_…
Content-Type: application/json

{ "noticeId": "…", "text": "Coffee time!", "env": { "t": 21.4, "rh": 38, "p": 1012, "iaq": 41
},
  "battery": 82, "photo": false }
```

Server:

```
        await Log.create({
          userId: req.user.id,
          event: 'device_copy',
          text: body.text,
          metadata: { deviceId, serial, noticeId, env, battery, photoUrl },
          context: await getLogContext(req.user),
        })
```

and add `'device_copy'` (and `'device_env'`) to `displayableEvents` so it appears in the Log tab.

3.2 Notice generation ("Coffee time!")

New scheduled job (in `scheduled-jobs.ts`), every 15 min for users with a paired device:

1. Run `generateContextualPrompts()` with current weather, hour, recent check-ins.
2. If nothing scores ≥ 6, ask the AI engine (`aiEngineManager`) for one short, kind, time-of-day notice ≤ 84 chars ("Coffee time!", "Rain at 4 pm — take the umbrella", "Stand up, stretch").
3. Limits: max 6 notices / day, none in quiet hours (user timezone), no repeats within 24 h.
4. Insert into `device_notices`. The device picks it up on its next poll.

4. Session compression (server side)

`POST /api/device/session` body is CBOR, `Content-Encoding: x-heatshrink`. Server decompresses, validates with a schema, then writes **one** compact Log row per session (`event: 'device_env'`, metadata = min/max/avg per sensor), not one row per sample. This keeps the Log tab readable and the DB small.

5. Settings UI (web)

Settings → **Devices**: "Add LOT Pager" (shows pair code), list of paired devices (battery, last seen, firmware), "Send test notice", "Unpair".

6. Implementation checklist (next sessions)

- ☐ Migration: `devices`, `device_notices`
- ☐ Models + `src/server/routes/device-api.ts` + register in `routes/index.ts` and `index.ts`
- ☐ Add `device_copy`, `device_env` to `displayableEvents` and Log tab renderer
- ☐ Scheduled notice job + AI prompt
- ☐ Settings → Devices UI
- ☐ Device simulator script (`scripts/hardware/device-sim.ts`) so the loop can be tested before hardware exists